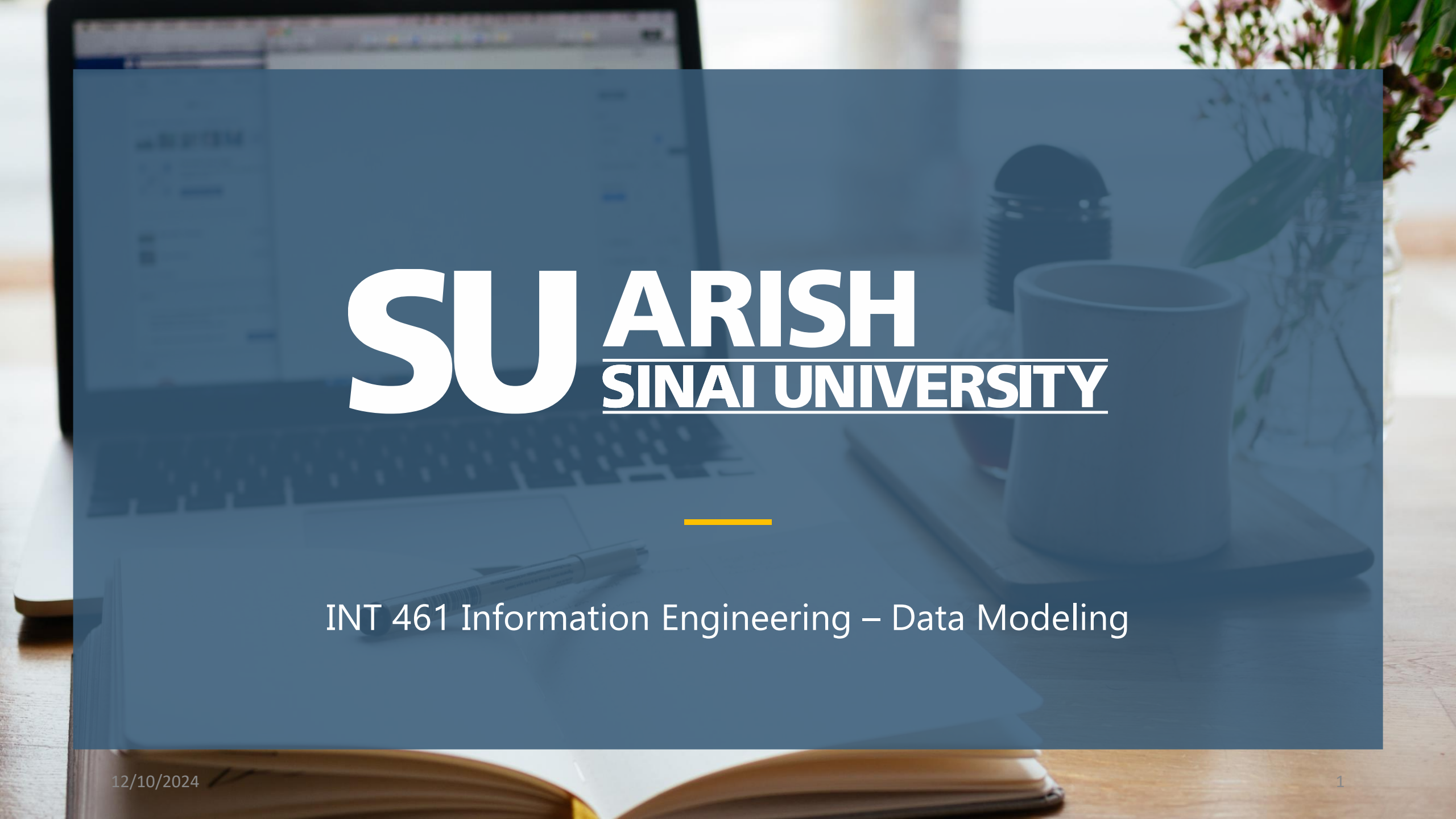


INT 461 Information Engineering – Data Modeling

INDEX

- Learning Objectives S. 3
- Introduction S.4
- Entities and Entity Class S.5
- Relationships S.14
- ER Diagram S15
- A Problem (5 marks) S38

@Sinaiunieg
info@su.edu.eg
www.su.edu.eg

Learning Objectives

- **Understand the steps of data modeling**
- **Extract entity classes**
- **Define data items of each entity class**
- **Identify key items of each entity class**
- **Define relations between entity classes**
- **Develop an ERD**

ENTITY- RELATIONSHIP MODEL: Introduction

- Need to have a Data Model: evolution of DB and RDB
- Chen idea
- Inspired ideas after ERD: Mind Maps, and Concept Maps.
- Features of ER modeling
  - Simple
  - Flexible
  - Easy to map to a relational DB
- Don’t mix up ERD and DB schema

Entities and Entity Class

- An **entity** is an object that exists and is distinguishable from other objects. It can be tangible or intangible.
  - Example: specific person, company, event, plant, idea
- Entities have **attributes**
  - Example: people have *names* and *addresses*
- An **entity set or entity class** is a set of entities of the same type that share the same properties.
  - Example: set of all persons, companies, trees, holidays

Entities and Entity Class

- We model entities of interest in the developed SW.

- For example, if developing a student registration system, then we expect entity classes: Student, Course, Teacher,…etc
- These are the entity classes of interest in this case.
- Determining the entity classes is a crucial activity when developing SW.

ATTRIBUTES

- An entity is represented by a set of attributes, that is descriptive properties possessed by all the members of an entity set.
  - Example:

    Student = (Student-Id, Name, Sex, Birthdate, …)

    instructor = (Teacher-*ID, name, address, salary*)
    course= (*course_id, title, credits*)

- **Domain** – the set of permitted values for each attribute
- For example: Sex may be Female or Male
- ID is an integer number

ATTRIBUTES

- **Attribute types:**
  - **From the structural point of view can be: Simple** or **composite** attributes.
  - **From the value point of view can be: Single-value** or **multivalue** attributes
  - **From the storage point of view can be: Stored or Derived** attributes

TYPES of ATTRIBUTES

Simple Attribute:

Attribute that consists of a single atomic value. Sometimes it is called atomic attribute. It could not be divided into meaningful parts. Example: Salary, sex,

Composite Attribute:

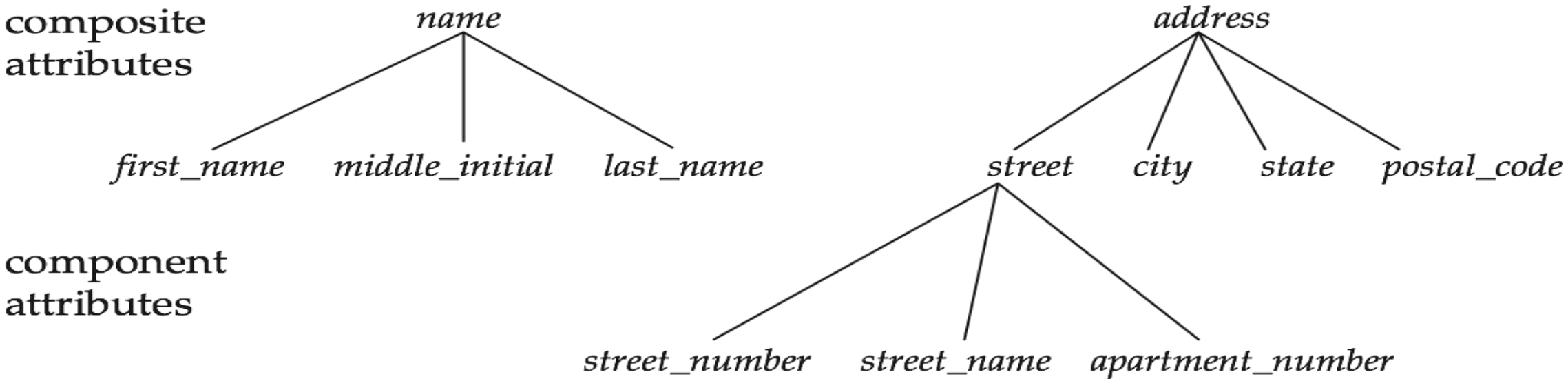
Attribute value not atomic. It can be divided into meaningful parts. Sometimes it is called grouped, compound, or complex attribute

Example : Address: "House _no, Street, City"

Name: "First Name: Middle Name: Last Name“

Note that It is possible that a particular attribute be atomic in one system and composite in another one. It depends upon the system itself and whether splitting an attribute to its parts (if it is not an atomic one) or keeping it as if it is an atomic one.

TYPES of ATTRIBUTES



TYPES of ATTRIBUTES

Single Valued Attribute: Attribute that holds a single value

Examples: Name, Sex, Id, Birth date, …..

Multi Valued Attribute: Attribute that holds multiple values.

Examples: Mobile no, email, ….

Again, it depends upon the analyst and the user requirements whether to define an attribute as a multi valued attribute or not. For example, the university may allow one email only for its students which will be used to communicate with them.

TYPES of ATTRIBUTES

Stored and Derived Attributes

Stored attributes are the attributes that their values are stored in the system. On the other hand, derived attributes are the attributes that their values are derived (calculated) from stored one (s) but not stored in the system.

Examples:

Age is a derived attribute and its value is calculated using the birth date which is a stored attribute.
Why?
Can you give another example from student system?

ENTITY SETS *INSTRUCTOR* AND *STUDENT*

instructor_ID instructor_name

76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

instructor

student-ID student_name

98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student

RELATIONSHIPS

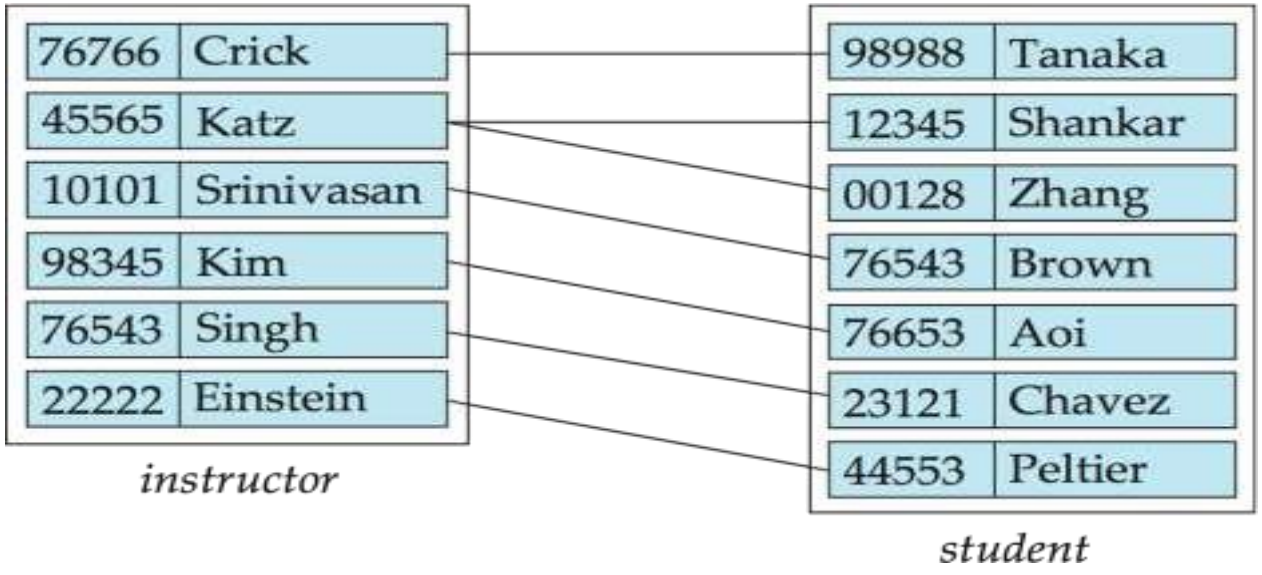
- A **relationship** is an association among several entities

  Example:

  44553 (Peltier) — *student* entity

  advisor — relationship set

  22222 (Einstein) — *instructor* entity



ENTITY-RELATIONSHIP DIAGRAMS

- Representing entities
  - we represent an entity by a named rectangle
  - use a singular noun, or adjective + noun
  - refer to one instance in naming

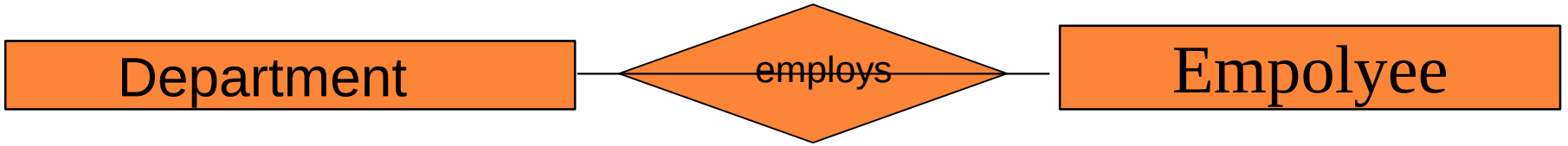
8

CUSTOMER

PART-TIME EMPLOYEE

ENTITY-RELATIONSHIP DIAGRAMS

- Representing relationship

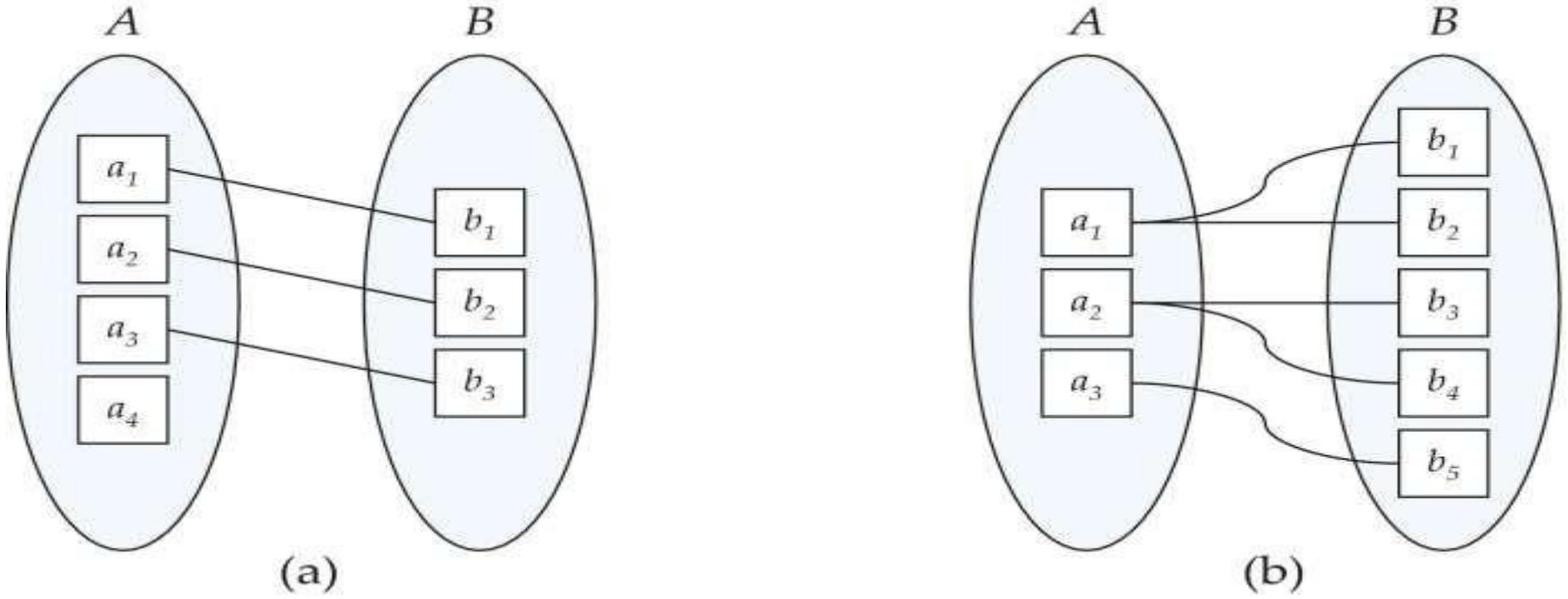


ENTITY-RELATIONSHIP DIAGRAMS

Types of Relationships

- Three types of relationships can exist between entities
- One-to-one relationship (1:1): One instance in an entity (parent) refers to one and only one instance in the related entity (child).
- One-to-many relationship (1:M): One instance in an entity (parent) refers to one or more instances in the related entity (child)

ENTITY-RELATIONSHIP DIAGRAMS

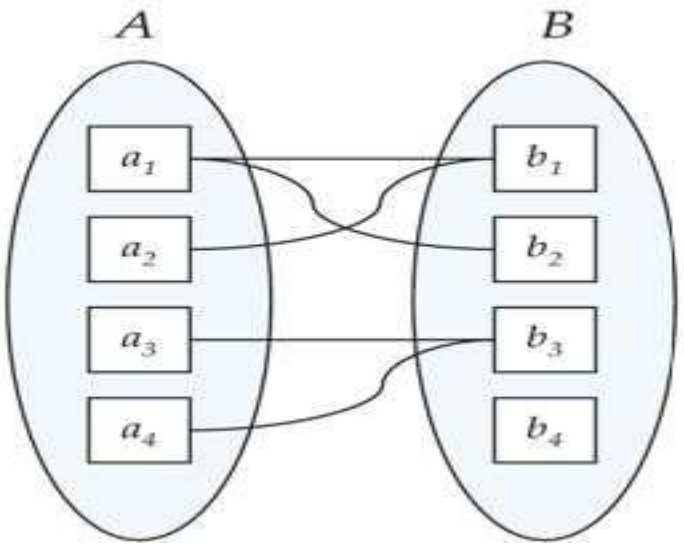


One to one

One to many

Entity-Relationship Diagrams

- Types of Relationships
  - Many-to-many relationship (M:N): exists when one instance of the first entity (parent) can relate to many instances of the second entity (child), and one instance of the second entity can relate to many instances of the first entity.



Many to many

CARDINALITY CONSTRAINTS

- We express cardinality constraints by drawing either a directed line (→), signifying "one," or an undirected line (—), signifying "many," between the relationship set and the entity set.
- Or, by numbering each entity. * or, m for many.

- One-to-one relationship:
  - A student is associated with at most one *instructor* via the relationship *advisor*
  - A *student* is associated with at most one *department* via *stud_dept*



ONE-to-MANY RELATIONSHIP

- one-to-many relationship between an *instructor* and a *student*
  - an instructor is associated with several (including 0) students via *advisor*
  - a student is associated with at most one instructor via advisor,



MANY-TO-MANY RELATIONSHIP

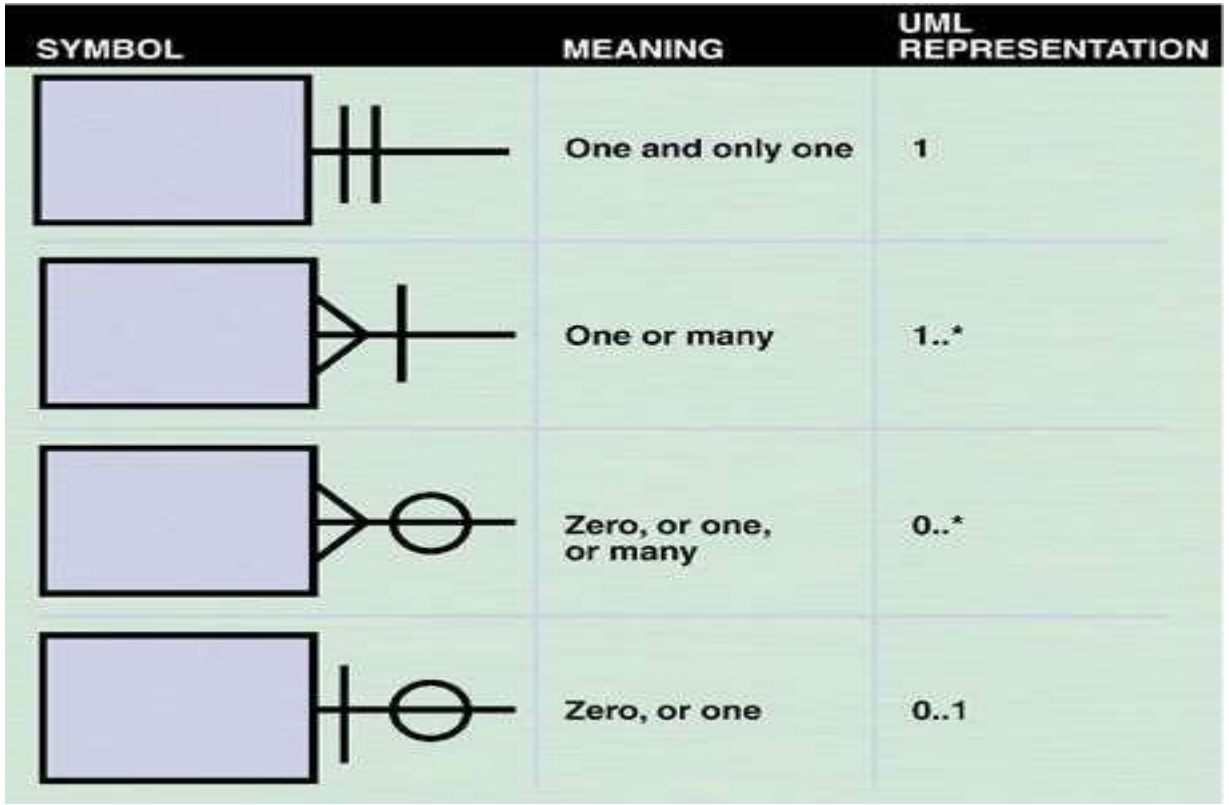
- An instructor is associated with several (possibly 0) students via *advisor*
- A student is associated with several (possibly 0) instructors via *advisor*



- Crow’s foot notation: A type of cardinality notation. It is called crow's foot notation because of the shapes, which include circles, bars, and symbols, that indicate various possibilities.

- A single bar indicates one, a double bar indicates one and only one, a circle indicates zero, and a crow's foot indicates many.

ENTITY-RELATIONSHIP DIAGRAMS



SYMBOL	MEANING	UML REPRESENTATION
	One and only one	1
	One or many	1..*
	Zero, or one, or many	0..*
	Zero, or one	0..1

17

Crow's foot notation is a common method of indicating cardinality. The four examples show how you can use various symbols to describe the relationships between entities.

DIFFERENT TYPES of KEYS

- A **candidate key** of an entity set is a minimal super key
  - *ID* is candidate key of *instructor*
  - *course_id* is candidate key of *course*



StudentId	firstName	lastName	courseId
L0002345	Jim	Black	C002
L0001254	James	Harradine	A004
L0002349	Amanda	Holland	C002
L0001198	Simon	McCloud	S042
L0023487	Peter	Murray	P301
L0018453	Anne	Norris	S042

PRIMARY KEY

- A primary key is a candidate key that is most appropriate to be the main reference key for the table. As its name suggests, it is the primary key of reference for the table and is used throughout the database to help establish relationships with other tables.
- **The primary key must contain unique values, must never be null and uniquely identify each record in the table**

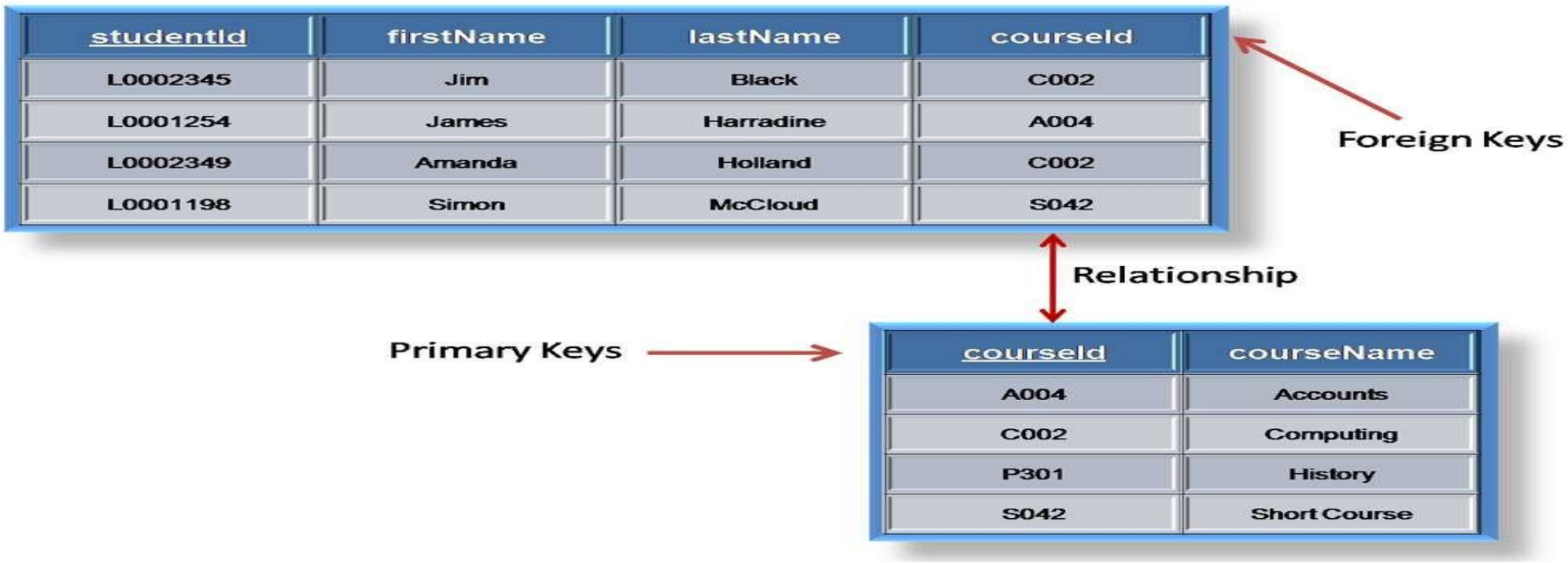
Primary Keys



StudentId	firstName	lastName	courseId
L0002345	Jim	Black	C002
L0001254	James	Harradine	A004
L0002349	Amanda	Holland	C002
L0001198	Simon	McCloud	S042
L0023487	Peter	Murray	P301
L0018453	Anne	Norris	S042

FOREIGN KEY

- A foreign key is generally a primary key from one table that appears as a field in another where the first table has a relationship to the second. In other words, if we had a table A with a primary key X that linked to a table B where X was a field in B, then X would be a foreign key in B



DIFFERENT TYPES of KEYS

A **super key** of an entity set is a set of one or more attributes whose values uniquely determine each entity.

Example:

- {Student ID,FirstName }
- {Student ID, LastName }
- {Student ID,FirstName,LastName}

ERD

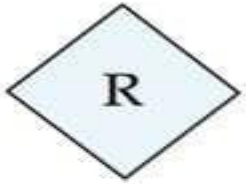


- Rectangles represent entity sets.
- Diamonds represent relationship sets.
- Attributes listed inside entity rectangle. Or , as oval shape along with the rectangle.
- Underline indicates primary key attributes

SUMMARY OF SYMBOLS USED IN E-R NOTATION



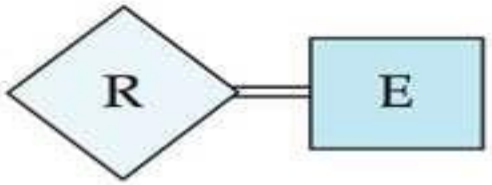
entity set



relationship set



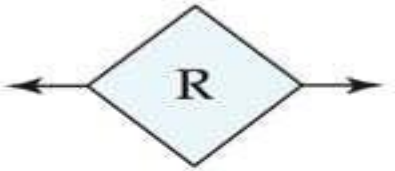
identifying relationship set for weak entity set



total participation of entity set in relationship



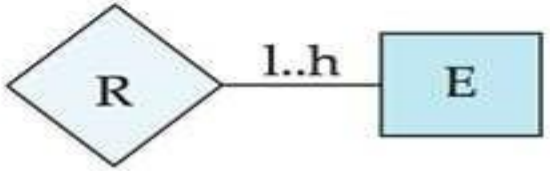
many-to-many relationship



one-to-one relationship



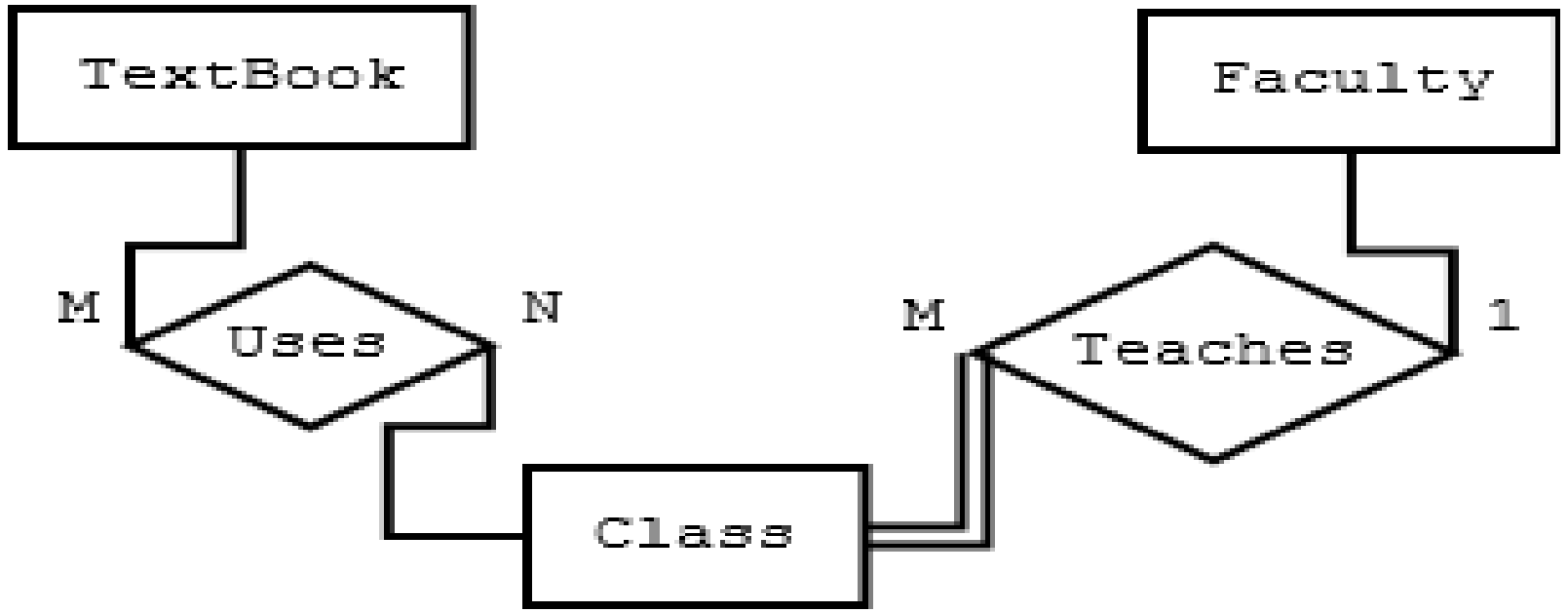
many-to-one relationship



cardinality limits

TOTAL PARTICIPATION of ENTITY SET

- E.g., A *Class* entity cannot exist unless related to a *Faculty* member entity



WEAK ENTITY SETS

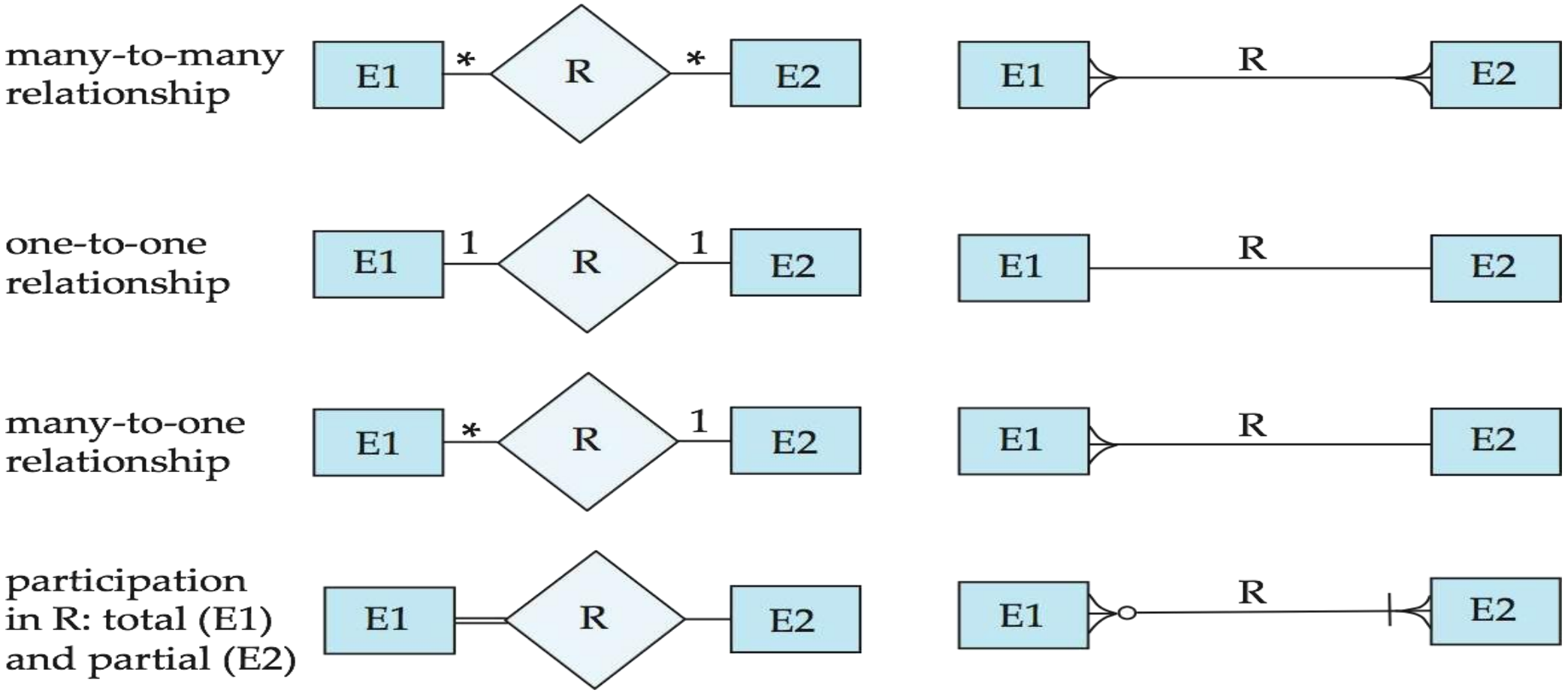
An entity set that does not have a primary key is referred to as a **weak entity set**

- We underline the discriminator of a weak entity set with a dashed line.
- We put the identifying relationship of a weak entity in a double diamond.
- Primary key for *section* – (*course_id, sec_id, semester, year*)



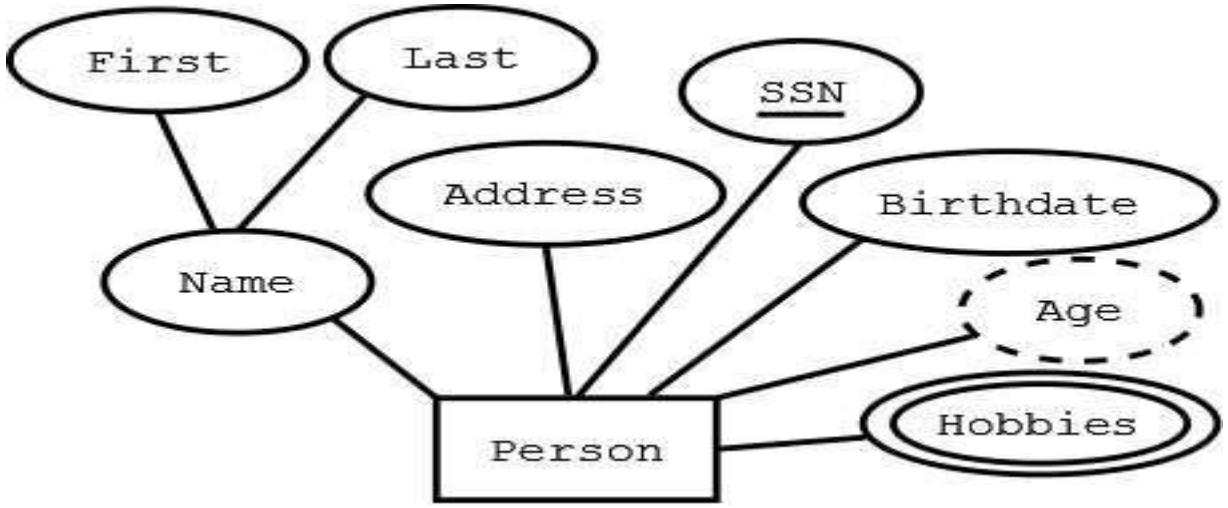
SUMMARY OF SYMBOLS USED IN E-R NOTATION

Crows feet notation



Summary of Symbols Used in E-R Notation

- Representing attributes



- **Rectangle** -- Entity
- **Ellipses** -- Attribute (underlined attributes are [part of] the primary key)
- **Double ellipses** -- multi-valued attribute
- **Dashed ellipses**-- derived attribute, e.g. age is derivable from birthdate and current date.

Summary of the Steps to do Data Modeling using the ER Approach

1- Identify entity classes (or sets)

2- Identify data elements in each entity class

3- Define/Identify keys of each entity class

4- Draw a draft of the ER model

5- Refine the ERD

Attributes of a good ERD

1- Simple
2- Non redundant
3- Flexible to cater or cope for future modifications
4- Preferably does not have loops nor weak entity classes

Loops: where some entities in an entity class are related to other entities in the same entity class.

A typical example is the entity class “Employee” and the relation “Manages”.

Attributes of a good ERD

Loops: where some entities in an entity class are related to other entities in the same entity class. A typical example is the entity class “Employee” and the relation “Manages”.
A weak entity class is an entity class which has no primary key itself but rather it depends on the key of a “parent” entity class. A typical example is the entity class “Employee” and the entity class “Dependents”.

A Problem (5 marks)

- It is required to develop an ERD for the university library. The thought system should keep data about books, journals, theses, newspapers, … etc
- Data about the borrower of books should be kept also. A staff member may borrow a book for 14 days, while a student can borrow a book for 7 days at most.
- You should assume reasonable attributes for your design.

THANK YOU

For any questions feel free
to contact me by mail

Nasser.tamim@su.edu.eg

